System Verification and Validation Plan for Software Engineering

Team 20, Aeolus Software
Andy Huynh
Andreas Sideris
Humna Saeed
Christopher Ludwig
Owen Johnson

November 3, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 2

3 Plan **3**
- 3.1 Verification and Validation Team . . . 3
- 3.2 SRS Verification . . . 3
- 3.3 Design Verification . . . 3
- 3.4 Verification and Validation Plan Verification . . . 4
- 3.5 Implementation Verification . . . 4
- 3.6 Automated Testing and Verification Tools . . . 4
- 3.7 Software Validation . . . 5

4 System Tests **6**
- 4.1 Tests for Functional Requirements . . . 6
  - 4.1.1 Area of Testing1 . . . 6
  - 4.1.2 Central Hub Tests for Functional Requirements . . . 7
  - 4.1.3 User Interface Tests for Functional Requirements . . . 11
- 4.2 Tests for Nonfunctional Requirements . . . 14
  - 4.2.1 Area of Testing1 . . . 14
  - 4.2.2 Central Hub Tests for Nonfunctional Requirements . . . 15
  - 4.2.3 User Interface Tests for Non-Functional Requirements . . . 16
- 4.3 Traceability Between Test Cases and Requirements . . . 19

5 Unit Test Description **20**

6 Appendix **21**
- 6.1 Symbolic Parameters . . . 21
- 6.2 Usability Survey Questions? . . . 22

7 Appendix — Generative AI Use Disclosure **23**
- 7.1 AI Tools Used . . . 23
- 7.2 How and Where Used . . . 23

7.3 Verification of AI-Generated Content 23

8 Appendix — Reflection **24**

List of Tables

1 RealmForge System Test Requirements Traceability Summary 19

2 Symbolic Constants Used in the Verification and Validation Plan 22

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS [1] tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

[Say what software is being tested. Give its name and a brief overview of its general functions. —SS]

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

The Custom Documents the project will involve are a Feature Oriented Domain Analysis Report and a Usability Report.

A feature oriented domain analysis is an analysis showing the similarities and differences for a family of software products. It identifies common requirements and variabilities between members of a family of products, with variation parameters for each variability. Tables and/or diagrams of features

can be used to display this.

A usability report is a systematic, rigorous and quantified report summarizing the results of usability testing plans. These results include metrics collected from game sessions, and user feedback about game design, issues/bugs, and user experience recommendations.

2.4 Relevant Documentation

- **Problem Statement and Goals** [2]: This document is relevant as it outlines the initial outline of RealmForge.
- **Development Plan** [3]: This document outlines the main goals, objectives, and stretch goals for the system. This informs all future development decisions, including this document.
- **System Requirements Specification (SRS)** [1]: This document outlines the functional and non-functional requirements for the system. The requirements listed inform the verification and validation efforts of this document.
- **Hazard Analysis** [4]: This document outlines safety and security requirements of the system, as well as listing the different potential failure modes of the system with causes and recommended action.
- **Feature Oriented Domain Analysis (FODA)** [**CITATION**]: This custom document outlines the variabilities within a software product line, which informs what common or unique aspects need to be verified in each member of the software family.
- **Usability Report** [**CITATION**]: This custom document outlines the results of usability testing. These results will be important when making any considerations about the system.
- **Module Guide (MG)** [5]: This document will be a reference for system components, informing important modules to test.
- **Module Interface Specification (MIS)** [6]: This document will help guide and define function, and therefore unit testing, in the system. This informs tests within the current document.

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Citation Example of SRS[1]

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]
[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.5 Implementation Verification

The implementation verification will ensure that the implementation of RealmForge's codebase adheres to the SRS specifications set out. A combination of unit testing, static analysis, and regular code inspections.

Unit tests will be carried out using GDUnit4 [CITATION] as listed in this document. Passing these unit tests will verify if functionality has changed in any unintended way during refactoring a PR is allowed to be merged into their parent branch.

Static analysis will be carried out by GDLint [CITATION] throughout development to ensure a consistent code quality and style (as described in the Development Plan [CITATION]).

All code written will be reviewed by at least one other team member in a routine code inspection before merging. This will ensure not only that the code is of proper quality, but will ensure that the team is all on the same page and nothing takes anyone by surprise.

[**You should at least point to the tests listed in this document and the unit testing plan.** —SS]

3.6 Automated Testing and Verification Tools

[What tools are you using for automated testing. Likely a unit testing framework and maybe a profiling tool, like ValGrind. Other possible tools include a static analyzer, make, continuous integration tools, test coverage tools, etc.

Explain your plans for summarizing code coverage metrics. Linters are another important class of tools. For the programming language you select, you should look at the available linters. There may also be tools that verify that coding standards have been respected, like flake9 for Python. —SS]

[If you have already done this in the development plan, you can point to that document. —SS]

[The details of this section will likely evolve as you get closer to the implementation. —SS]

3.7 Software Validation

The primary mechanism of validating RealmForge's software will be through extensive usability testing and playtesting, having users interact with the system, providing feedback according to our Usability Testing Plan found at **LINK THE PLAN**. Supplementary methods to usability testing and playtesting for software validation are:

- **Supervisor and Stakeholder Reviews:** Demonstrations of the project will be held throughout its lifecycle with course facilitators, key stakeholders (peers, potential users, etc.), and our project supervisor, Dr. Welch, such as during Proof of Concept and Rev0. We have also scheduled meetings to be held with Dr. Welch every other week to review anything in this project, including software validation.

- **Empirical Metric Validation:** Metrics (run duration, win/loss ratios, resource rates, etc.) will be collected with built-in tools will be collected during system use to create minimalistic quantitative date reports measuring engagement and other goals of the software. If tools allow, automated playthroughs of the game can be run to increase amount of empirical data.

- **Accessibility & Safety Validation:** Observational checks during playtesting alongside the execution of our UI system test suite will validate startup epilepsy warnings (**SSR-006**), colourblind-accessible filters (**SSR-009**). Local save data recovery (**SSR-003**, **SSR-004**) will be validated through our functional testing.

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

Control: Manual versus Automatic

Initial State:

Input:

Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

4.1.2 Central Hub Tests for Functional Requirements

The system tests below are planned to verify the implemented system as a whole against functional requirements related to the Central Hub component of the system.

1. **SFT-2.1** Permanent Upgrades to Central Hub

   Control: Manual

   Initial State: The current game state is focused on the Central Hub with a permanent upgrade available to be obtained.

   Input: The user unlocks the permanent upgrade, and the current run is then ended.

   Output: A permanent upgrade to the Central Hub is now associated with the corresponding player profile which unlocked it, and is always available to that profile in future runs.

   Test Case Derivation: This system test verifies that upgrades which are meant to be permanent are maintained between runs and are not unintentionally removed from the player profile.

   How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in a run, they will trigger a permanent upgrade unlock and then end the run either by intentionally exiting or dying. If the upgrade is supposed to take effect immediately, the system tester shall continue their run to see it's effect and then end the run. They will then start

a new run and see if the previously unlocked upgrade is still unlocked and working as intended.

2. **SFT-2.2** Resource Generation from Central Hub

   Control: Manual

   Initial State: The RealmForge software has been launched on the user's device and initialized into a game state where they are starting a combat encounter.

   Input: The user selects to enters the combat encounter and performs any action that requires resources.

   Output: The system initializes resource count variables to the starting amount corresponding with Central Hub progress, updating it in accordance to the resource generation rate of the Central Hub and maintaining an accurate count as user actions expend resources simultaneously. This resource count should be constantly displayed to the user.

   Test Case Derivation: This test verifies an essential mechanic of the core gameplay loop of RealmForge, ensuring that resource generation from the Central Hub works as intended, giving the user access to the correct number of resources in accordance to upgrades to the Central Hub.

   How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in the run, they will note down the current state of the Central Hub and calculate the correct amount of starting resources and the rate of generation of each resource type. Then the system tester shall enter a combat encounter, verifying the starting resource count against the calculated number as well as recording the rate at which they gain resources precisely to verify against the other calculated numbers.

3. **SFT-2.3** Temporary Upgrades to Central Hub

   Control: Manual

Initial State: The RealmForge software has been launched on the user's device and the current game state is initialized to focus on the Central Hub with a temporary upgrade available to be obtained.

Input: The user obtains the temporary upgrade and continues playing the game until their run ends and then begins a new run.

Output: The Central Hub is upgraded for the rest of the run, altering how it functions according to the upgrade selected, and then is reinitialized with none of the temporary upgrades when a new run is started after the initial run has ended.

Test Case Derivation: This system test verifies that temporary upgrades do affect how the Central Hub functions as well as not being initially present in the Central Hub when a new run is started.

How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in a run, they will trigger a temporary upgrade unlock and then continue their run, tracking that the upgrade to the Central Hub has altered it in exactly the way intended for the rest of the run. Then the system tester will end the run either by intentionally exiting or dying. They will then start a new run and see if the previously unlocked upgrade is no longer unlocked initially.

4. **SFT-2.4** Selecting Upgrades for the Central Hub

   Control: Manual

   Initial State: The RealmForge software has been launched on the user's device and the system is initialized to a game state where the user should be able to upgrade the Central Hub.

   Input: No input is necessary once at the initial state

   Output: The system shall always display atleast one upgrade to the user and all upgrades shall contained detailed descriptions for exactly what they do and the cost to obtain them.

   Test Case Derivation: This system test verifies the user will never get stuck with no options for upgrades to select and all upgrades always

contain the necessary details for the user to make informed decisions and leading to a better gameplay experience.

How test will be performed: The system tester shall play through many runs and track whether the intended output for displaying upgrade options is displayed as well as recording what they think each upgrade does according to the description and if they ever thought the cost of an upgrade was different from what it was.

5. **SFT-2.5** Rejecting Upgrade Unlock Due to Insufficient Funds

   Control: Manual

   Initial State: The RealmForge software has been launched on the user's device and the system is initialized to a game state where the user should be able to upgrade the Central Hub.

   Input: The user attempts to unlock an upgrade in which they do not meet the requirements for unlocking.

   Output: The system rejects the unlock attempt and notifies the player they do not meet the requirements for unlocking the upgrade

   Test Case Derivation: The system test ensures that the upgrade unlock mechanic is working as intended and does not allow users to unlock all upgrades for free, eliminating all need for thought and strategy for upgrade selection.

   How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in the run, they will play until reaching a game state where they can unlock upgrades for the Central Hub and one of the available upgrades is one in which they do not meet the requirements for. They will then attempt to unlock that upgrade and note down how the system responds.

6. **SFT-2.6** Verifying Game State Transition to the Central Hub

   Control: Manual

Initial State: The RealmForge software has been launched on the user's device and the current game state has the option to switch to the game state focused on the Central Hub.

Input: The user triggers the game state transition to focus on the Central Hub.

Output: The system transitions from one game state to the next, verifying a game state change did occur.

Test Case Derivation: The system test verifies that game state transitions are working in our system and bugs will not lock users out of progressing forward.

How test will be performed: The system tester will launch the RealmForge software, and begin a run or resume an existing one. Once they are in a run, they will play the game normally but they should keep close track of game information. When a game state transition should occur, the tester will verify the state change did happen and that no data was corrupted during the transition.

4.1.3 User Interface Tests for Functional Requirements

The system tests listed below are planned to verify the implemented system as a whole against functional requirements related to its user interface component. These requirements can be found in the SRS [1], listed in section S.2's functional requirements for the 'User Interface System', as well as in section S.2's Safety and Security Requirements. Traceability relationships between all system tests and covered requirements are provided in the traceability table in section 4.3 of this document.

1. **SFT-3.1** Main Menu Interface

   Control: Manual

   Initial State: The RealmForge software has been launched to its initial menu on a compatible device.

   Input: Each available main menu option is selected ('start game', 'settings', and 'exit') via an input device (keyboard and mouse, or a

controller). Once a main menu option has been selected, an input is also provided to return to the main menu from these options' screens (game start menu, configurations menu, etc.).

Output: The system transitions its internal state and displayed visuals to the selected menu after each selection input. For example, when 'settings' is selected, the display transitions to the configurations menu, and when an input is received to return to the main menu, the system transitions back to the main menu. This is also performed for other options available from the main menu.

Test Case Derivation: This system test verifies at a basic level that the system's main user interface (the main menu) can successfully receive user inputs and transition between states as necessary. This functionality is required for the system and its interface to receive other inputs and provide the necessary outputs during gameplay and other uses of the software.

How test will be performed: The system tester will launch the RealmForge software, and verify that the system successfully transitions the display output between each menu provided by the options available from the main menu, and back to the main menu when prompted. This will be verified for each major type of input device (keyboard and mouse, and controller).

2. **SFT-3.2** Game Session Restart Interface

   Control: Manual

   Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the player's current game run has ended (a run is an attempt at completing the game, which can end following the player successfully completing the game or failing during an attempt).

   Input: The various options available to the user following the end of a game run are selected (including restarting the run in the same saved game session / save file, or starting a new save file) via an input device (keyboard and mouse, or a controller).

Output: When the option to restart the game run in the same save file is selected, the system transitions its internal state and display output to the initial game scene displayed during a game run, with the user's previous save data included where appropriate. Similarly, when the option to start a new game run in a new save file is selected, the system transitions to the interface used to create a new save file.

Test Case Derivation: This system test verifies that the interface provided by the system following the end of a game run can successfully receive user inputs and transition between its next possible states as necessary (restarting in the same save file, or creating a new save file). This functionality is required for the system to allow users to continuously attempt runs of the game (which is a common gameplay loop for video games in the roguelite genre, as RealmForge is), or create a new save file when needed.

How test will be performed: The system tester will launch the RealmForge software, and configure it to enter a game state following the end of a player's game run. The tester will then verify that the system successfully transitions the display output between each interface provided by the options available following the end of a run (restarting in the same save file, and creating a new save file). This will be verified for each major type of input device (keyboard and mouse, and controller).

3. **SFT-3.3** Display of Flashing Lights Warning

   Control: Manual

   Initial State: The RealmForge software has been installed and deployed/compiled on a compatible device.

   Input: The RealmForge software is launched on a compatible device.

   Output: Following launch, the system displays a warning to the user that gameplay may result in the display of various images and colours that can cause seizures for individuals with photosensitive epilepsy (despite meeting standard guidelines, as outlined in SNFT-3.3). The warning clearly states that users with photosensitive epilepsy or a family

history of the condition should consult their physician before playing the game.

Test Case Derivation: This system test verifies that the system successfully displays a required warning for users with photosensitive epilepsy (or those with a family history of the condition), to ensure that these users do not continue gameplay without awareness of how it could affect their condition. While RealmForge is required to meet certain accessibility standards to ensure it does not display any flashing images that could harm these individuals (as outlined in SNFT-3.3), it is important to recommend that they consult their physician first before playing the game in the interest of caution.

How test will be performed: The system tester will launch the RealmForge software, and verify that the correct flashing images/lights warning is displayed by the system.

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

4.2.2 Central Hub Tests for Nonfunctional Requirements

The system tests below are planned to verify the implemented system as a whole against nonfunctional requirements related to the Central Hub component of the system.

1. **SNFT-2.1** Sufficient Variety to Upgrades in Central Hub

   Type: Manual

   Initial State: The RealmForge software has been launched on the user's device and they have started a new run or resumed an existing one.

   Input/Condition: The user plays through multiple runs of the game, unlocking temporary and permanent upgrades from the Central Hub as they see fit.

   Output/Result: The user will have their Central Hub upgraded and will report their opinion on the variety of upgrade options given to them.

How test will be performed: The system tester shall play through many runs of the game, noting down the number of upgrade options available to them whenever they are give the option to unlock upgrades. They should also be recording how different and impactful each upgrade seems to help measure variety between upgrades.

2. **SNFT-2.2** Available and Unavailable Upgrade Test for Central Hub

   Type: Manual

   Initial State: The RealmForge software has been launched on the user's device and the system is initialized to a game state where the user should be able to upgrade the Central Hub.

   Input/Condition: No input is required for this test

   Output/Result: The user will report if they were ever confused whether an upgrade as available for upgrade or not.

   How test will be performed: The system tester shall launch the RealmForge software and begin a new one or resume an existing one. They will play through these runs and every time an upgrade is available, they should confirm if their first thought whether an upgrade was available or not, recording every instance in which they were wrong.

4.2.3 User Interface Tests for Non-Functional Requirements

The system tests listed below are planned to verify the implemented system as a whole against nonfunctional requirements related to its user interface component. These requirements can be found in the SRS [1], listed in section S.2's nonfunctional requirements (they are spread across multiple categories), as well as in section S.2's Safety and Security Requirements (some safety requirements presented in the document relate to nonfunctional qualities of the system). Tracebility relationships between all system tests and covered requirements are provided in the traceability table in section 4.3 of this document.

1. **SNFT-3.1** Game World Location Appearance Variety

   Type: Dynamic, Manual

Initial State: The RealmForge software has been launched on a compatible device, and initialized to a game state where the overall game world is displayed during a gameplay session.

Input/Condition: The game world is traversed using inputs to the software as a player would during gameplay (assuming the software is initialized to a state where the player is permitted to move to each world location), using a valid input device (keyboard and mouse, or a controller).

Output/Result: The total game world displays at least DISTINCT_LOCATION_ICONS unique icons/symbols for the locations players can traverse to (these can be repeated across the game world as needed). A used symbol is distinct from others if it is visibly differentiated by colour (following required accessibility guidelines), or the images displayed (a mountain vs. a forest, for example).

How test will be performed: The system tester will launch the RealmForge software, and initialize its state to the overall game world (where each game world location is valid for travel). The tester will then traverse the game world using a valid input device as a player would, and verify that at least DISTINCT_LOCATION_ICONS distinct location symbols are displayed in the total game world, following the definition of distinct described above.

2. **SNFT-3.2** Game State Display Transitions

   Type: Dynamic, Manual

   Initial State: The RealmForge software has been launched on a compatible device, in its development environment.

   Input/Condition: The game is initialized to each generalized state it can enter (main menu, configuration menu, save data management menu, game world, and combat encounter) and each potential scene/menu transition for each state is executed via a valid input device (entering the configuration menu from the main menu, for example).

   Output/Result: The system completes and displays each prompted game scene transition in under TRANSITION_TIME seconds. During

each transition, the system fades the display output to black (or another suitable intermediate screen), to communicate that the transition is occurring.

How test will be performed: The system tester will launch the RealmForge software, and initialize its state to each generalized state it can enter (as discussed above). The tester will then prompt the system to execute each possible scene/menu transition from each state, and verify that each transition elapses at most TRANSITION_TIME seconds. This includes one transition for each time the game can transition to a new overall state (i.e. from the main menu to the save data management menu); granular transitions are not tested (such as the system transitioning from one state of an enemy encounter to another).

The tester will also verify that the system displays a valid transition screen, as described above.

Since this test involves verifying qualities of many different transitions the system can execute, it is suitable for each transition to be verified a minimum of TRANSITION_TESTS times. This ensures that the test is feasible in the context of the project.

3. **SNFT-3.3** Review of Flashing Lights in Game Content

   Type: Static, Manual

   How test will be performed: The RealmForge software's source code will be reviewed by the RealmForge developers to ensure that any graphical displays or changes follow the WCAG 2.2 accessibility guidelines' guideline 2.3.1 [7]. This guideline mandates that presented content does not display any flashing images more than three times in any one second period, to reduce any risks to individuals with photosensitive epilepsy.

   Each module of the RealmForge software's source code will be verified against this guideline, during regular reviews held by the RealmForge development team following the completion of a new section of source code. Sections of source code that display graphical outputs to the user in a timely fashion will be specifically reviewed, to ensure that they do not display any excessively bright images that change more than three times in one second.

4. **SNFT-3.4** Review of Distinguishable Colours in Game Content

   Type: Static, Manual

   How test will be performed: The RealmForge software's source code will be reviewed by the RealmForge developers to ensure that any graphical indicators or interface elements/buttons used in the software follow the WCAG 2.2 accessibility guidelines' guideline 1.4.1 [7]. This guideline mandates that "color is not used as the only visual means of conveying information, indicating an action, prompting a response, or distinguishing a visual element" [7].

   Each module of the RealmForge software's source code will be verified against this guideline, during regular reviews held by the RealmForge development team following the completion of a new section of source code. Sections of source code that display graphical outputs to the user that must be differentiated from other graphics (such as interface elements, system messages, etc.) will be specifically reviewed, to ensure that they provide a means of distinguishing these elements from others without relying on using colour.

4.3 Traceability Between Test Cases and Requirements

Table 1: RealmForge System Test Requirements Traceability Summary

System Test ID	Requirements Coverage
SFT-3.1	SFR-018
SFT-3.2	SFR-019
SFT-3.3	SSR-006
SNFT-3.1	SNFR-004
SNFT-3.2	SNFR-009
SNFT-3.3	SSR-006
SNFT-3.4	SSR-009
SFT-2.1	SFR-025
SFT-2.2	SFR-026

Requirement ID	Requirements Coverage
SFT-2.3	SFR-026
SFT-2.4	SFR-027, SFR-028, SFR-029
SFT-2.6	SSR-012
SNFT-2.2	SSR-008
SFT-2.5	SFR-030
SNFT-2.1	SNFR-001

5 Unit Test Description

TBD

References

[1] Aeolus Software (https://github.com/Aeolus-Software), "Realmforge system requirements specification," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/SRS/SRS.pdf, Aeolus Software, 2026.

[2] ——, "Realmforge problem statement and goals," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/ProblemStatementAndGoals/ProblemStatement.pdf, Aeolus Software, 2026.

[3] ——, "Realmforge development plan," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf, Aeolus Software, 2026.

[4] ——, "Realmforge hazard analysis," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, Aeolus Software, 2026.

[5] ——, "Realmforge module guide," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, Aeolus Software, 2026.

[6] ——, "Realmforge module interface specification," https://github.com/Aeolus-Software/RealmForge/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf, Aeolus Software, 2026.

[7] World Wide Web Consortium (W3C), "How to Meet WCAG (Quick Reference): A Customizable Quick Reference to Web Content Accessibility Guidelines (WCAG) 2 Requirements," *W3C*, https://www.w3.org/WAI/WCAG22/quickref/, (accessed Nov. 1, 2026).

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The following symbolic constants are used throughout this document:

Table 2: Symbolic Constants Used in the Verification and Validation Plan

Constant	**Value**	**Unit**
DISTINCT_LOCATION_ICONS	5	Unitless/Quantity
TRANSITION_TIME	4	Seconds
TRANSITION_TESTS	3	Unitless/Quantity

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

7.1 AI Tools Used

- Anthropic Claude Sonnet 5.5
- Google Search AI Overview (Google Gemini)

7.2 How and Where Used

AI tools were used for assistance with LaTeX syntax and commands, during the development of sections SRS Verification and Design Verification, and when developing LaTeX formatting for the document as a whole. Specifically, AI tools were used for assistance in creating the LaTeX custom system test format macro used throughout the document, as well as the symbolic parameter table used in section 6.1. In these sections, AI tools were not used to generate any content text (LaTeX formatting code was generated), or wholly generate any of the ideas or conclusions presented.

7.3 Verification of AI-Generated Content

Any LaTeX syntax or code provided by AI tools was verified by testing compilation in LaTeX documents, and ensuring the resulting output was correctly formatted and matched the needs of the document. No ideas or text (outside of LaTeX formatting code) were newly generated by AI tools and presented in the document.

8 Appendix — Reflection

Reflection Questions Answered:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Andreas:

1. When writing this deliverable, our team worked well to divide the work required evenly, and ensure that all team members contributed to each important section of the document. This helped us to ensure that critical sections were developed by taking all team members' ideas and contributions into account. Specifically, instead of having one team member focus on writing a large swath of system tests, and making other members focus on most of the general plan sections of the document (SRS verification, etc.), all users contributed to the system tests, ensuring that our ideas and perspectives were combined to cover the system's requirements as much as possible.

2. One pain point I experienced during the deliverable was attempting to find a balance between feasibility and coverage when writing system tests. When creating a plan to test the system as a whole, our goal was to cover all requirements mandated in the SRS, and extensively test the system so that it is verified and validated as a professional engineering product. However, since the project's development must fit in an eight month scope (of which there are approximately five to six

months remaining), with verification activities likely being allotted a fraction of that time (to fit with other development activities), the plan for testing must be scoped to be feasible in the project's context. This balance was found by ensuring that the system tests I wrote minimally cover the requirements provided in the SRS, and any other system qualities that must be tested, but do not necessarily have to extend into a large, complex set of tests across a large sample size. For example, while primary playtests (usability tests) for the software are planned, the number of participants in these tests and the amount of testing they must participate in is given a conservative bound, to ensure feasibility. However, if extra time and resources are available in the project to complete more testing, the scope of the tests planned can be increased to ensure better coverage.

3. One skill I will personally need to acquire is a proficiency with unit testing Godot game engine code. Since Godot has been selected as the game engine that will be used for the development of the project's software, its capabilities for unit testing its GDScript language must be used to implement automated unit testing. Additionally, I will need to acquire the skill of executing automated unit tests of GDScript code with GitHub Actions or CI/CD pipelines, since it is required for the project.

4. For both skills above, the knowledge required to improve my proficiency could be found from video tutorials (on YouTube, for example), official Godot tutorials and documentation, or from being taught by other team members or peers with experience with Godot. In my case, I will primarily use the official Godot tutorials and documentation to learn these skills, as official sources tend to have the most accurate information for the exact version of the game engine we will use in the project. Additionally, not relying on others for instruction related to the skills (while help can still be acquired from team members or peers occasionally) can help me to efficiently learn the skills required asynchronously.

Humna:

1.

2.

3.

4.

Christopher:

1.

2.

3.

4.

Owen:

1.

2.

3.

4.

Andy:

1.

2.

3.

4.